

4주차 - hovi

React에서 useRef는 어떤 용도로 사용하나요?

- 카테고리: React

답변

useRef는 렌더링과 무관하게 값을 유지하는 목적으로, 크게 두 가지 용도로 사용됩니다.

첫째, DOM 요소에 직접 접근할 때 사용합니다. ref 객체를 JSX의 ref 속성에 연결하면 해당 DOM 노드를 `.current` 로 참조할 수 있어, 포커스 제어나 스크롤 이동 같은 명령형 조작이 가능합니다.

둘째, 리렌더링 없이 값을 저장하고 싶을 때 사용합니다. useState와 달리 `.current` 값이 바뀌어도 컴포넌트가 다시 렌더링되지 않으므로, 타이머 ID나 이전 상태 값 등을 저장하는 데 적합합니다.

꼬리질문

- useRef와 useState의 차이는 무엇인가요?
- useRef로 이전 상태 값을 저장하는 패턴은 어떻게 구현하나요?

자바스크립트에서 this는 어떻게 결정되나요?

- 카테고리: JavaScript

답변

this는 함수가 선언된 위치가 아니라 **호출 방식**에 따라 동적으로 결정됩니다.

일반 함수 호출에서는 전역 객체(브라우저에서 `window`, strict mode에서는 `undefined`)를 가리킵니다. 메서드 호출에서는 `.` 앞의 객체를 가리킵니다. `call`, `apply`, `bind` 를 사용하면 this를 명시적으로 지정할 수 있습니다. `new` 키워드와 함께 호출하면 새로 생성된 인스턴스 객체가 this가 됩니다.

화살표 함수는 예외적으로 자신만의 this를 갖지 않고, 선언된 위치의 상위 스코프의 this를 그대로 사용합니다.

꼬리질문

- 화살표 함수에서 this가 다르게 동작하는 이유는?
- call, apply, bind의 차이는?

해시 테이블이란 무엇인가요?

- 카테고리: 자료구조

답변

해시 테이블은 키-값 쌍을 저장하는 자료구조로, 해시 함수를 통해 키를 인덱스로 변환하여 데이터를 저장하고 조회합니다.

평균적으로 삽입, 삭제, 검색 모두 $O(1)$ 의 시간 복잡도를 가져 빠른 접근이 가능합니다. 하지만 서로 다른 키가 같은 인덱스에 매핑되는 해시 충돌이 발생할 수 있으며, 이를 해결하기 위해 체이닝(연결 리스트)이나 개방 주소법(다른 빈 슬롯 탐색) 등의 방법을 사용합니다.

자바스크립트에서는 `Object` 와 `Map` 이 해시 테이블 기반으로 구현되어 있습니다.

꼬리질문

- 해시 충돌이 발생하면 어떻게 해결하나요?
- Object와 Map의 차이는 무엇인가요?

HTTP/1.1과 HTTP/2의 차이점은 무엇인가요?

- 카테고리: Network

답변

HTTP/2는 HTTP/1.1의 성능 한계를 해결하기 위해 설계되었습니다.

HTTP/1.1은 하나의 TCP 연결에서 요청-응답을 순차적으로 처리하여 Head-of-Line Blocking이 발생합니다. 여러 리소스를 동시에 받으려면 다수의 TCP 연결을 열어야 했습니다.

HTTP/2는 하나의 TCP 연결 위에서 여러 요청과 응답을 동시에 주고받는 멀티플렉싱을 지원합니다. 또한 헤더 압축(HPACK), 서버 푸시, 스트림 우선순위 설정 등의 기능이 추가되어 페이지 로딩 속도가 크게 개선됩니다.

꼬리질문

- 멀티플렉싱이란 무엇인가요?
- HTTP/3은 어떤 점이 다른가요?

큐와 스택의 차이는 무엇인가요?

- 카테고리: 자료구조

답변

스택과 큐는 데이터를 넣고 빼는 순서가 반대인 선형 자료구조입니다.

스택은 LIFO(Last In, First Out) 구조로, 가장 마지막에 들어온 데이터가 먼저 나갑니다. 함수 콜 스택, 브라우저 뒤로가기, 실행 취소(Undo) 등에 활용됩니다.

큐는 FIFO(First In, First Out) 구조로, 먼저 들어온 데이터가 먼저 나갑니다. 이벤트 큐, BFS 탐색, 프린터 대기열 등에 활용됩니다.

자바스크립트에서 스택은 배열의 `push / pop`, 큐는 `push / shift` 로 구현할 수 있습니다.

꼬리질문

- 자바스크립트의 콜 스택은 어떻게 동작하나요?
- 우선순위 큐란 무엇인가요?

웹 스토리지(localStorage, sessionStorage)와 쿠키의 차이는?

- 카테고리: Browser

답변

세 가지 모두 브라우저에 데이터를 저장하지만, 용량, 수명, 서버 전송 여부에서 차이가 있습니다.

쿠키는 약 4KB로 용량이 작고, `Set-Cookie` 헤더로 설정되며, 매 HTTP 요청마다 서버에 자동 전송됩니다. 만료 시간을 설정할 수 있습니다.

localStorage는 약 5~10MB로 용량이 크고, 브라우저를 닫아도 데이터가 유지되며, 서버에 자동 전송되지 않습니다.

sessionStorage는 localStorage와 비슷하지만 탭이나 창을 닫으면 데이터가 삭제됩니다.

인증 토큰은 쿠키(HttpOnly 설정), 사용자 설정은 localStorage, 일시적 상태는 sessionStorage에 저장하는 것이 일반적입니다.

꼬리질문

- HttpOnly 쿠키란 무엇이고 왜 사용하나요?
- localStorage에 민감한 정보를 저장하면 안 되는 이유는?

FSD 아키텍처란 무엇인가요?

- 카테고리: React

답변

FSD(Feature-Sliced Design)는 프론트엔드 프로젝트를 계층(Layer)과 슬라이스(Slice)로 나누어 구조화하는 아키텍처 방법론입니다.

app, processes, pages, widgets, features, entities, shared 의 7개 계층으로 구성되며, 상위 계층은 하위 계층만 참조할 수 있다는 단방향 의존성 규칙을 따릅니다. 각 계층 안에서는 도메인별로 슬라이스를 나누고, 슬라이스 내부는 ui, model, api, lib 등의 세그먼트로 구성됩니다.

기능 단위로 코드가 응집되어 있어 특정 기능을 수정할 때 관련 코드를 한곳에서 찾을 수 있고, 의존성 방향이 명확해 대규모 프로젝트에서 유지보수성이 높아집니다. 다만 소규모 프로젝트에서는 구조가 과할 수 있습니다.

꼬리질문

- FSD에서 feature와 entity의 차이는 무엇인가요?
- FSD의 단방향 의존성 규칙을 위반하면 어떤 문제가 생기나요?

CSS의 box-sizing 속성은 무엇인가요?

- 카테고리: CSS

답변

box-sizing은 요소의 너비와 높이를 계산할 때 padding과 border를 포함할지 결정하는 속성입니다.

content-box (기본값)는 width/height가 콘텐츠 영역만을 의미하며, padding과 border가 추가되어 실제 크기가 더 커집니다. border-box 는 width/height에 padding과 border를 포함해 지정한 크기 그대로 유지됩니다.

실무에서는 크기 계산이 직관적인 border-box 를 전역으로 설정하는 것이 일반적입니다.

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

꼬리질문

- content-box에서 padding을 추가하면 레이아웃이 깨지는 이유는?
- box-sizing이 inline 요소에도 적용되나요?

SSR(서버 사이드 렌더링)과 CSR(클라이언트 사이드 렌더링)의 차이는?

- 카테고리: React

답변

SSR은 서버에서 HTML을 완성하여 보내고, CSR은 브라우저에서 자바스크립트로 화면을 그리는 방식입니다.

CSR은 초기에 빈 HTML과 JS 번들을 받아 클라이언트에서 렌더링하므로 초기 로딩이 느리지만, 이후 페이지 전환이 빠릅니다. SEO에 불리하고 First Contentful Paint가 늦습니다.

SSR은 서버에서 완성된 HTML을 보내므로 초기 콘텐츠 표시가 빠르고 SEO에 유리합니다. 하지만 서버 부하가 높고, 인터랙션을 위해 hydration 과정이 필요합니다.

Next.js는 페이지별로 SSR, SSG, CSR을 혼합하여 각 장점을 활용할 수 있게 합니다.

꼬리질문

- Hydration이란 무엇인가요?
- SSG(Static Site Generation)와 SSR의 차이는?

XSS(크로스 사이트 스크립팅)란 무엇이고 어떻게 방어하나요?

- 카테고리: Network

답변

XSS는 공격자가 웹 페이지에 악성 스크립트를 삽입하여 사용자의 브라우저에서 실행시키는 보안 공격입니다.

Stored XSS는 악성 스크립트가 서버에 저장되어 다른 사용자가 페이지를 열 때 실행됩니다. Reflected XSS는 URL 파라미터 등에 포함된 스크립트가 응답에 반영되어 실행됩니다. DOM-based XSS는 클라이언트 자바스크립트가 사용자 입력을 검증 없이 DOM에 삽입할 때 발생합니다.

방어 방법으로는 사용자 입력을 HTML 이스케이프 처리하고, CSP(Content Security Policy) 헤더를 설정하며, 쿠키에 HttpOnly 플래그를 사용합니다. React에서는 JSX가 기본적으로 이스케이프 처리를 해주지만, `dangerouslySetInnerHTML` 사용 시 주의해야 합니다.

꼬리질문

- CSP(Content Security Policy)란 무엇인가요?
- React에서 `dangerouslySetInnerHTML`을 안전하게 사용하는 방법은?

Controlled 패턴과 Uncontrolled 패턴을 함께 사용하는 경우는 있나요?

- 카테고리: React

답변

네, 하나의 폼 안에서 두 패턴을 혼합하여 사용하는 경우가 있습니다.

대부분의 입력 필드는 Controlled로 관리하면서, 파일 입력(`input type="file"`)처럼 리액트로 값을 제어할 수 없는 요소는 Uncontrolled로 두는 것이 일반적입니다. `react-hook-form` 같은 라이브러리는 내부적으로 Uncontrolled 방식을 사용하면서 `register` 와 `watch` 로 필요한 시점에만 값을 읽어, Controlled의 편리함과 Uncontrolled의 성능 이점을 결합합니다.

핵심은 각 필드의 요구사항에 맞게 적절한 패턴을 선택하는 것이며, 반드시 하나로 통일할 필요는 없습니다.

async와 defer 속성의 차이는 무엇인가요?

- 카테고리: Browser

답변

둘 다 HTML 파싱을 차단하지 않고 스크립트를 비동기로 다운로드하지만, **실행 시점과 순서 보장**이 다릅니다.

`async` 는 다운로드가 완료되는 즉시 실행하며, 실행 중에는 HTML 파싱이 중단됩니다. 여러 스크립트 간 실행 순서가 보장되지 않으므로 독립적인 스크립트(광고, 분석 도구)에 적합합니다.

`defer` 는 다운로드를 병렬로 하되, HTML 파싱이 모두 끝난 후에 실행합니다. 선언 순서대로 실행이 보장되므로 DOM에 의존하는 스크립트에 적합합니다. `DOMContentLoaded` 이벤트 직전에 실행됩니다.

연결 리스트(Linked List)와 배열 기반 리스트의 차이는 무엇인가요?

- 카테고리: 자료구조

답변

메모리 구조와 연산 성능에서 차이가 있습니다.

배열 기반 리스트는 연속된 메모리에 데이터를 저장하여 인덱스로 $O(1)$ 접근이 가능하지만, 중간 삽입·삭제 시 요소를 이동시켜야 해 $O(n)$ 이 소요됩니다.

연결 리스트는 각 노드가 데이터와 다음 노드의 주소(포인터)를 가지며 메모리상 흩어져 있습니다. 중간 삽입·삭제가 $O(1)$ 이지만 특정 위치 접근은 처음부터 순회해야 하므로 $O(n)$ 입니다.

접근이 잦으면 배열, 삽입·삭제가 잦으면 연결 리스트가 유리합니다.

TLS 핸드셰이크 과정에서 대칭키와 비대칭키는 어떻게 함께 사용되나요?

- 카테고리: CS

답변

TLS 핸드셰이크에서는 비대칭키 암호화로 안전하게 대칭키를 교환하고, 이후 실제 데이터 통신은 대칭키로 수행합니다.

클라이언트가 서버에 연결을 요청하면, 서버는 공개키가 포함된 인증서를 보냅니다. 클라이언트는 인증서를 검증한 뒤, 세션에 사용할 대칭키(또는 대칭키 생성에 필요한 값)를 서버의 공개키로 암호화하여 전송합니다. 서버는 개인키로 복호화하여 대칭키를 얻습니다.

이후 통신은 속도가 빠른 대칭키 암호화로 진행됩니다. 비대칭키의 안전한 키 교환과 대칭키의 빠른 암호화, 두 장점을 결합한 방식입니다.

이진 탐색 트리(BST)의 시간 복잡도는 항상 $O(\log n)$ 인가요?

- 카테고리: 자료구조

답변

아니요, BST의 시간 복잡도는 트리의 균형 여부에 따라 달라집니다.

균형이 잘 잡힌 BST에서는 탐색, 삽입, 삭제가 모두 $O(\log n)$ 입니다. 하지만 데이터가 정렬된 순서로 삽입되면 트리가 한쪽으로 치우쳐(skewed) 사실상 연결 리스트와 같은 형태가 되어, 최악의 경우 $O(n)$ 까지 성능이 저하됩니다.

이 문제를 해결하기 위해 AVL 트리나 레드-블랙 트리 같은 자가 균형 이진 탐색 트리(Self-Balancing BST)가 사용됩니다. 이들은 삽입·삭제 시 자동으로 트리를 회전시켜 균형을 유지하므로, 항상 $O(\log n)$ 성능을 보장합니다.